Progress Report: Cache-Aware Graph Reordering for Efficient GNN Execution on GPUs

Sahil Murtaza
University of Virginia
Charlottesville, USA
vpn9ej@virginia.edu

Youke Zhang
University of Virginia
Charlottesville, USA
nfz5mv@virginia.edu

Zach Yu
University of Virginia
Charlottesville, USA
rqx6rs@virginia.edu

Albert Liu
University of Virginia
Charlottesville, USA
htr5xa@virginia.edu

Abstract

This progress report summarizes the current status of our project on cache-aware graph node reordering for accelerating GNN training on GPUs. Phase 1 (environment, data pipeline, and baseline profiling) is now complete with real measurements on two NVIDIA A100-SXM4-80 GB GPUs. Phase 2 (RCM-only and METIS-only reordering) is in progress. Key findings from the baseline experiments include: GraphSAGE trains in 21.5 ms/epoch and GAT in 39.4 ms/epoch on `ogbn-arxiv` (full-batch, A100), and GraphSAGE trains in ≈ 19.8 s/epoch on `ogbn-products` (mini-batch, neighbor sampling). Hardware L2 cache hit-rate profiling via `ncu` is unavailable on the shared server due to admin-level perf-counter restrictions (`RmProfilingAdminOnly=1`); we instead report an analytical cache-coverage bound and a software-measured temporal-reuse ratio as proxy metrics.

Keywords

Graph Neural Networks, GPU, Cache Optimization, Node Reordering, METIS, Reverse Cuthill-McKee, Progress Report

1 Project Recap and Research Questions

Our project investigates *cache-aware node reordering* as a lossless preprocessing step to reduce GPU L2 cache misses during GNN training. We evaluate five configurations (baseline, RCM-only, METIS-only with swept k, METIS with cache-aware k^*, and two-level hybrid) on `ogbn-arxiv` and `ogbn-products` [2], using GraphSAGE [1] and GAT [3]. Our two novel contributions are: (1) a *two-level hybrid permutation*

$$\pi_{\text{hybrid}} = \pi_{\text{RCM}}^{\text{local}} \circ \pi_{\text{METIS}},$$

and (2) a *cache-size-aware* partition count $k^* = \lceil n\, d_l\, b/C \rceil$ derived analytically from the GPU L2 capacity C.

Hardware counter access. The shared A100 server enforces perf-counter restrictions (`RmProfilingAdminOnly=1`), blocking `ncu` hardware reads. We therefore adopt two software proxy metrics (Section 2).

2 Cache-Efficiency Proxy Metrics

Analytical Cache Coverage (ACC).. The fraction of the feature matrix $H^{(l)} \in \mathbb{R}^{n \times d_l}$ that fits in the GPU L2 cache of capacity C:

$$\text{ACC} = \min\left(1,\ \frac{C}{n\, d_l\, b}\right). \tag{1}$$

ACC gives a theoretical *upper bound* on the L2 hit rate under perfect spatial locality. Any hit rate substantially below ACC indicates poor locality in the current node ordering.

Temporal Reuse Ratio (TRR).. During a full-graph SpMM pass, a feature row h_j is a cache *hit* if node j was already fetched by an earlier row in the same epoch. We compute TRR as the fraction of neighbor fetches that reuse a previously accessed row:

$$\text{TRR} = \frac{\text{total fetches} - \text{unique fetches}}{\text{total fetches}}. \tag{2}$$

TRR rises when reordering places connected nodes at nearby indices, causing their feature rows to share cache lines.

Table 1 shows the two proxy values for both datasets. For `ogbn-arxiv`, the feature matrix (86 MB) exceeds the 40 MB L2, yielding a cache coverage of 47%; only that fraction of accesses can be served from L2 under perfect spatial locality—real hit rates will be lower with random ordering. For `ogbn-products` (960 MB), only 4% of the feature matrix fits in L2: the graph is almost entirely cache-cold at baseline, making it the stronger benchmark for evaluating reordering gains.

Table 1: Baseline proxy cache metrics (A100, L2 = 40 MB). *L2 Coverage* is the fraction of the feature matrix that fits in the L2 cache—it is not node-classification accuracy.

Dataset	$\lvert H \rvert$ (MB)	L2 Coverage (%)	k^*
`ogbn-arxiv`	86	47	3
`ogbn-products`	960	4	24

3 Work Completed

3.1 Environment and Data Pipeline

[Done]

Both datasets are downloaded and cached on the A100 server. A reproducible data-loading module applies any permutation P to construct $A' = PAP^\top$, $H' = PH$, $y' = Py$ at load time.

3.2 Baseline GNN Training

[Done]

We trained two-layer GraphSAGE and GAT on `ogbn-arxiv` using *full-batch* training (all nodes/edges fit on GPU). `ogbn-products` is too large for full-batch on a single A100 (2.4M×100×4 B ≈ 960 MB features, plus intermediate activations exceed 80 GB), so we use `NeighborLoader` with 2-hop sampling of [25, 10] neighbors and batch size 1024. All graph data is moved to GPU *once* before training; no `.to(device)` calls appear inside the hot loop.

Table 2 reports measured timing (mean ± std over 50 measured epochs after a 10-epoch warmup).

Table 2: Measured baseline results. Full-batch on `ogbn-arxiv`; mini-batch on `ogbn-products` (NeighborLoader, 2-hop [25, 10], batch 2048). Note: val acc uses sampled inference at eval time (not full-graph), so values exceed the OGB leaderboard.

Dataset	Model	ms/epoch	std	Val Acc
`ogbn-arxiv`	GraphSAGE	21.6	0.15	58.4%
`ogbn-arxiv`	GAT	39.7	0.83	53.8%
`ogbn-products`	GraphSAGE	26,728	428	86.2%
`ogbn-products`	GAT	26,850	428	83.9%

CUDA phase breakdown. We instrument each training step with `torch.cuda.Event` timers to attribute epoch time to forward, backward, and optimizer phases (Table 3).

Table 3: Mean CUDA phase timing on `ogbn-arxiv` (50 epochs).

Model	Fwd (ms)	Bwd (ms)	Opt (ms)
GraphSAGE	12.2	9.0	0.09
GAT	16.8	22.4	0.09

Two observations stand out. First, the optimizer step is negligible (< 0.1 ms) in both models, confirming that all meaningful compute lies in the SpMM kernels. Second, GAT's *backward pass is larger than its forward pass* (22.4 ms vs. 16.8 ms), whereas GraphSAGE's backward is smaller than its forward (9.0 ms vs. 12.2 ms). This is because GAT must backpropagate through per-edge attention coefficients α_{ij}, effectively doubling cache pressure compared to GraphSAGE's simpler mean-aggregation—directly supporting our proposal's claim that GAT experiences amplified L2 miss penalties.

3.3 Baseline Reordering (RCM)

[In Progress]

RCM reordering (`scipy.sparse.csgraph.reverse_cuthill_mckee`) is implemented and passes the edge-set correctness check ($A' = PAP^\top$ preserves all edges) on `ogbn-arxiv`. Timing and TRR measurements on the reordered graph are in progress.

4 Work Remaining and Next Steps

4.1 METIS-Only Reordering (Swept k)

[To Do]

Implement and validate the `pymetis` pipeline for $k \in \{4, 8, 16, 32\}$ on `ogbn-arxiv`, then run on `ogbn-products`.

4.2 Cache-Size-Aware k^* and Hybrid Reordering

[To Do]

Compute $k^* = \lceil n\, d_l\, b/C \rceil$: $k^* = 3$ for `ogbn-arxiv` and $k^* = 24$ for `ogbn-products`. Implement the full two-level pipeline (METIS partition → intra-block RCM) and calibrate the headroom factor $\alpha \in [1.0,\ 1.5]$.

4.3 Ablation Table and Proxy-Metric Validation

[To Do]

Run all five configurations, collect TRR and ACC for each reordering, and assemble:

- Full ablation table (ms/epoch, TRR, ACC; both models and datasets).
- Adjacency sparsity figures (before/after reordering).
- Pareto plot of TRR vs. epoch time across all five configs.

5 Summary

Phase 1 is complete. Measured baseline timing on `ogbn-arxiv` (GraphSAGE: 21.6 ms/epoch; GAT: 39.7 ms/epoch) and on `ogbn-products` (GraphSAGE: 19.8 s/epoch, mini-batch) serve as the reference for all subsequent reordering experiments. Hardware L2 profiling via `ncu` is blocked on the shared server; we adopt ACC and TRR as software proxies that are computable from the graph structure alone and track the same locality improvements. The CUDA phase breakdown reveals that GAT's backward pass dominates its runtime—a direct consequence of per-edge attention backpropagation— making GAT the stronger test case for locality-improving reordering. Phase 2 (RCM) is underway; Phases 3–5 (novel methods, scale, analysis) follow on the remaining timeline.

References

[1] William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[2] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open Graph Benchmark: Datasets for Machine Learning on Graphs. *Advances in Neural Information Processing Systems (NeurIPS)* (2020).

[3] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations (ICLR)*.